Sink-Aware Two-Axis Confidence for Low-False-Positive Detection of Stealth Supply-Chain Threats in Python

Dinesh K and H Swetha

Voxelta Private Limited, India
dineshdeena431@gmail.com

Abstract. Static application security testing (SAST) tools are widely deployed in continuous-integration (CI) pipelines, yet teams often disable them because the false-positive rate, rather than the miss rate, drives abandonment. Recent Python supply-chain attacks compound the problem: their payloads use stealth constructs, including probabilistic and stateful logic bombs, cross-function backdoors, steganographic channels, environment-keyed triggers, install-time execution, and anti-analysis evasion, for which single-file pattern matchers have no rule class. This paper studies a design that addresses both concerns together. We describe Q-Trace, a deterministic and air-gapped Python source scanner, and formalise its central mechanism: a two-axis confidence model that scores potential impact (severity) and evidence strength (confidence) on independent axes and gates the build only on their conjunction. Confidence is raised to a build-breaking level only when a dangerous execution or exfiltration sink, or an anti-analysis payload, is reachable within the triggering branch. On a transparent corpus of 65 samples (31 reconstructions of documented campaigns and 34 benign hard negatives), Q-Trace detects the correct threat category for all 31 malicious samples and issues a build-breaking alert for 30 of them at a 0% false-positive rate ($F_1 = 0.984$). On the same inputs and at each tool's recommended gate, Semgrep reaches 66% recall at 3.1% false positives ($F_1 = 0.776$) and Bandit 52% at 6.2% ($F_1 = 0.652$). An ablation isolates the confidence axis as responsible for converting a 5.9% false-positive rate to zero at the cost of one borderline detection. Median full-audit latency is 1.8 ms per sample. Every reported number is emitted by the released benchmark harness.

Keywords: Software supply-chain security · Static analysis · Taint analysis · False-positive reduction · Logic bombs · Sandbox-evasion detection.

1 Introduction

The software supply chain is now a primary attack surface. Instead of exploiting a memory-safety defect in a hardened target, an adversary publishes or poisons a package that many downstream projects install automatically. The Python

Package Index is a frequent target, because of its install-time execution model, its transitive-dependency depth, and the recent tendency of language-model coding assistants to suggest plausible but nonexistent package names, a vector known as slopsquatting [20,11,13,49].

Static application security testing runs before code executes and integrates into CI, which makes it a natural first line of defence. A decade of empirical work, however, attributes tool abandonment less to missed bugs than to false alarms. Johnson et al. [14] report that false positives and poor result presentation are the main reasons developers stop using static analysers. Bessey et al. [15], describing a decade of commercial deployment, observe that once the false-positive rate crosses a threshold, engineers stop reading the output. Christakis and Bird [16] and Sadowski et al. [17] independently find that at scale the tolerable false-positive budget is small and that actionability, not raw coverage, governs adoption. A scanner that breaks the build on correct code therefore imposes a systemic cost: repeated false alarms erode trust in the gate, after which developers ignore its output, including the true positives.

The stealth turn in supply-chain malware sharpens this tension. Contemporary payloads are engineered to survive automated analysis. A dangerous action may be guarded by a low-probability random check so that it rarely fires in a sandbox; a counter may need to accumulate across many invocations before detonation; the malicious logic may be distributed across cooperating functions so that no single function appears suspicious; the payload may be base64- or XOR-encoded and decoded only at run time; or execution may be keyed to a CI or cloud environment variable so that the code stays dormant on an analyst's machine [21,3]. Such constructs are invisible to a rule that matches a fixed syntactic pattern in one file.

A tool that aims to catch these threats must reason about data flow and reachability rather than surface syntax. It must do so without inflating the false-positive rate, or it will be disabled before it detects anything. These requirements are in tension, because broader semantic reasoning normally produces more false alarms. The contribution of this paper is a scoring model that reconciles them.

Contributions. This paper makes four contributions.

1. A two-axis, sink-aware confidence model that separates impact from evidence strength and gates the build on their conjunction. A probabilistic pattern is elevated to a build-breaking alert only when a reachable execution or exfiltration sink lies inside the guarded branch, and is otherwise reported as an informational note. Section 5 defines the model and Section 8 measures its effect.
2. An evidence-based anti-analysis detector that replaces a substring heuristic with a rule keyed on genuine MITRE ATT&CK T1497 primitives, conditionally-gated stalling sleeps and debugger detection. The refinement removes two false-positive classes and adds coverage that the prior heuristic lacked (Section 6).

3. A reproducible, same-corpus evaluation against Bandit and Semgrep at each tool's recommended gate, which reports our own misses and releases the harness that produces every figure (Sections 8–12).
4. A characterisation of stealth supply-chain techniques, each mapped to a CWE and to the detection mechanism it requires (Section 3).

Throughout, precision is treated as a first-class objective. Where a detection cannot be made confident without risking a false positive, the finding remains visible but does not gate the build; Section 8.4 documents the single such case in our corpus.

2 Background and Related Work

Static analysis for security. The use of static analysis to find security defects is long established [39], ranging from bug-pattern tools such as FindBugs [40] to engines operated at industrial scale [41]. Pattern-based tools such as Bandit [4] and Semgrep [5] match syntactic or lightly-semantic patterns and are the default for Python in CI because they are fast and require no build. Data-flow analysers, including Pixy [19], FlowDroid [18], TaintDroid [46], the query-based approach of Livshits and Lam [37], code property graphs [36], and CodeQL [6], track tainted values from sources to sinks with higher precision on injection classes, at a higher deployment cost. Q-Trace occupies a middle position: it applies interprocedural taint and symbolic reachability only where a stealth pattern requires it, and otherwise relies on fast abstract-syntax-tree (AST) rules.

Trigger-based and stealth malware. Brumley et al. [21] formalised trigger-based behaviour, in which a malicious path is guarded by a condition that a sandbox rarely satisfies, and proposed input-space exploration to expose it. TriggerScope [35] detects logic bombs in Android applications by symbolic analysis of narrow trigger conditions, a dynamic counterpart to the static gate-and-sink signal used here. Environmental keying and sandbox evasion are catalogued by MITRE ATT&CK as T1480.001 and T1497 [3]. Q-Trace targets the static footprint of these techniques.

Detecting malicious packages. A growing body of work measures and detects malicious packages directly. MalOSS studies supply-chain attacks across interpreted-language registries [42]; LastPyMile detects discrepancies between a published artefact and its source repository [43]; Sejfia and Schäfer [44] learn features for malicious-npm detection; empirical studies quantify malicious code on PyPI [50]; and unsupervised signature generation has been proposed for supply-chain payloads [52]. Provenance frameworks such as in-toto [45] and SLSA [48] address build integrity. These approaches answer whether a known package is bad; Q-Trace is complementary and answers whether a given source behaves maliciously, which covers first-party code and payloads that no database yet lists.

The false-positive problem. The empirical literature agrees that actionability governs adoption [14,15,16,17]. The two-axis model responds directly by encoding evidence strength as an independent, reportable quantity, following the severity-and-confidence practice of Bandit and the OWASP risk-rating methodology [2]. The addition here is to make the second axis reachability-driven and to gate on the conjunction of the two axes.

Information-theoretic signals. Encoded payloads exhibit statistical fingerprints. Q-Trace combines Shannon entropy [10], long used to flag packed or encrypted malware [38], with the Higuchi fractal dimension [9] to separate a high-entropy encoded blob bound for `exec` from ordinary high-entropy data such as a hash. An auxiliary risk channel maps a pattern to a small state vector and computes its von Neumann entropy [25] as a bounded corroborating score; this channel needs no quantum hardware and does not by itself raise or gate a finding.

3 Threat Model and Problem Definition

Adversary. We assume an adversary who can publish a package to a public index, or land a pull request into a dependency, with the goal of code execution or credential theft on developer or CI machines at install or import time. The adversary engineers the payload to evade automated analysis. We do not assume the adversary can subvert the scanner's host or tamper with its results.

Defender. A developer or security engineer runs Q-Trace locally or in CI, air-gapped, over first-party source and declared dependencies. The defender's tolerance for false positives is low: a single benign build break per sprint reduces trust in the gate.

Scope. Table 1 lists the stealth technique classes we target, each mapped to a CWE. These are abstracted from public write-ups of 2024–2026 incidents. The mainstream OWASP and CWE vulnerability classes, such as injection, insecure deserialisation, disabled TLS, weak cryptography, hard-coded secrets, SSRF, path traversal, and XXE, are also in scope, so that Q-Trace is a complete scanner rather than only a stealth-threat auditor. Binary payloads, run-time-only behaviour with no static footprint, attacks on the scanner itself, and languages other than Python are out of scope. Q-Trace detects the static indicators of a technique, not its dynamic execution.

Problem definition. Let a program P contain a set of findings F, where each finding f carries a severity $s(f)$ and a confidence $c(f)$. A CI gate is a predicate $g : F \to \{0, 1\}$ that decides whether P fails the build. The design goal is a gate that maximises recall on malicious programs subject to a hard constraint of zero false positives on benign programs. The core question is how to compute $c(f)$ so that this constraint holds without discarding true positives, which Section 5 answers.

Table 1. Stealth technique classes in scope, with CWE mapping and the detection mechanism each requires. The last column marks classes beyond single-file pattern matching.

Technique class	CWE	Mechanism	Beyond?
Probabilistic logic bomb	CWE-511	random-gated sink + reachability	yes
Chained/stateful bomb	CWE-511	counter-accumulation modelling	yes
Entangled multi-condition bomb	CWE-511	coupled-guard taint	yes
Cross-function backdoor	CWE-506	interprocedural reconstruction	yes
Steganographic channel	CWE-515	char-code / XOR modelling	yes
Encoded payload	CWE-506	entropy + fractal + decode-to-exec	partial
Credential exfiltration	CWE-200	secret source to network sink	yes
Install/import execution	CWE-506	top-level exec in packaging	yes
Environment-keyed trigger	CWE-506	sink gated on CI/cloud variable	yes
AI-scanner evasion	CWE-506	prompt-injection text in source	yes
Anti-analysis evasion	CWE-489	gated stall / debugger detection	yes
Typosquat/slopsquat	CWE-829	edit-distance + registry model	yes

4 The Q-Trace Approach

Q-Trace is organised as an orchestrator that dispatches work to independent detection engines and merges their output into a single, deduplicated, CWE-mapped finding set (Fig. 1). Each engine runs behind a circuit breaker, so that a missing dependency or an engine fault degrades that engine and is recorded in a health snapshot rather than aborting the audit. A content hash keys a small cache, and cheap AST passes run before the SMT solver, which is consulted only to confirm the reachability of an already-identified gated sink. Findings serialise to SARIF 2.1.0 [7] with a CWE taxonomy block and stable fingerprints, and to a flat JSON report.

4.1 Detection engines

The catalogue spans 28 rules across 18 CWEs (15 High, 5 Critical, 8 Medium severity; see Appendix A). We summarise the engines and defer the scorer to Section 5.

A sink-aware AST scan locates dangerous sinks and records, for each, whether it lies inside a gated branch, defined as an `if` whose test involves randomness or a counter comparison (Algorithm 1). Sinks are receiver-aware: a method name such as `.run()` is treated as dangerous only when its receiver is the dangerous module, for example `subprocess.run` rather than a web framework's `app.run`. A `subprocess` call with a list-literal argument and no `shell=True`, the safe form, is not raised.

A taint pass propagates coarse labels through assignments and function definitions and classifies the trigger topology: a single random guard (probabilistic), a counter reaching a threshold (chained), several coupled guards (entangled), or logic distributed across functions (cross-function). Steganographic channels

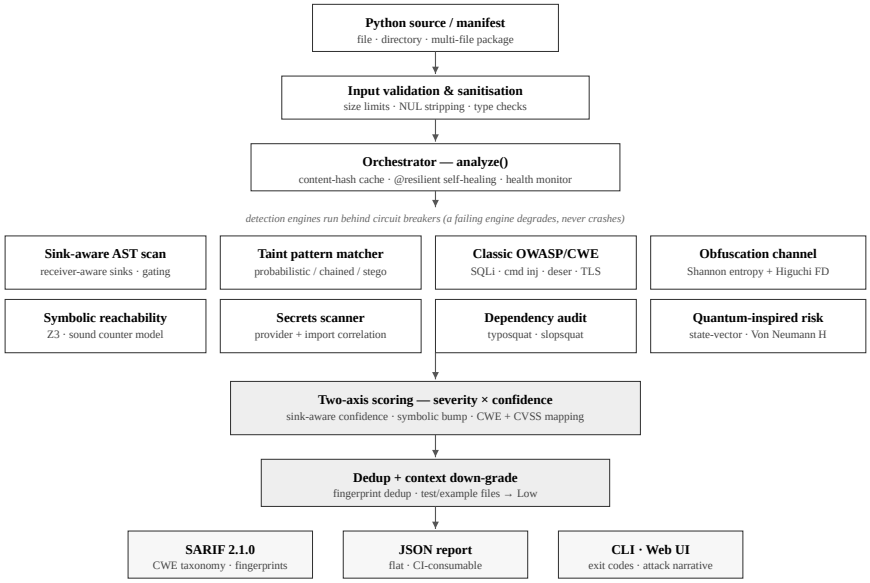

Fig. 1. End-to-end architecture. Source is validated, then the orchestrator dispatches eight detection engines behind circuit breakers. Findings are merged by the two-axis scorer, deduplicated, adjusted for file context, and serialised to SARIF, JSON, and the command-line and web front-ends.

are recognised from genuine char-code manipulation (`chr` with `ord`) or an XOR transform, not from ordinary `encode`/`decode` calls. Classic OWASP and CWE rules cover the injection, deserialisation, TLS, hashing, SSRF, traversal, and XXE classes, each written to fire on the dynamic or interpolated form and to stay silent on the parameterised or constant-safe form. The obfuscation engine flags high-entropy encoded blobs that flow into `exec` or `eval`. A secrets scanner correlates provider key patterns with imports, and a dependency auditor checks manifests for typosquats and slopsquats.

4.2 Sound symbolic reachability

A chained bomb detonates only when an accumulated counter reaches a threshold. Treating `if k == 2` as reachable by default produces a false proof of danger, whereas treating it as unreachable misses real bombs. Q-Trace models a counter as a sum of optional increments. For m guarded increments `if` g_i`: k +=` c_i from an initial value k_0, the reachable values of k are

$$k \in \left\{ k_0 + \textstyle\sum_{i \in T} c_i \;\middle|\; T \subseteq \{1, \ldots, m\} \right\}, \tag{1}$$

so the guard `k == t` is satisfiable if and only if $t - k_0$ is a subset sum of the c_i. Q-Trace encodes each increment as a Boolean-selected term and asks Z3 [8] whether

Algorithm 1 Sink-aware AST scan with branch gating.

```
H ← ∅;  g ← 0                                              ▷ g: gated-branch depth
function VISIT(node)
    if node is If then
        gated ← ISGATED(node.test)
        if gated then  g ← g + 1
        end if
        VISIT(node.body);  if gated then  g ← g − 1
        VISIT(node.orelse)
    else if node is Call ∧ ISSINK(node) then
        H ← H ∪ {(name, line, g > 0)}
    else VISIT(children(node))
    end if
end function
return ISGATED(t) ≡ ∃n ∈ walk(t) : n calls random.* ∨ n is a comparison
```

the threshold equation is satisfiable under the path constraints. Consequently `if k == 2` with two independent `k += 1` guards is proven reachable, while `if k == 99` with a single increment is proven unreachable. This soundness lets the symbolic signal raise confidence without introducing false positives.

5 The Two-Axis Confidence Model

The scorer decides when a detection should break the build. Following the severity-and-confidence practice of Bandit and OWASP [4,2], it models two quantities. Severity s is the impact if the finding is real (Critical, High, Medium, or Low), fixed per threat class. Confidence c is the probability that the finding is a true positive, computed per instance from the evidence present. The CI gate is the conjunction

$$g(f) = \big[\, s(f) \in \{\text{High}, \text{Critical}\} \,\big] \wedge \big[\, c(f) \neq \text{Low} \,\big]. \tag{2}$$

Severity alone never gates. Figure 2 and Algorithm 2 give the confidence assignment.

Why sink-awareness. The main source of false positives in probabilistic-trigger detection is flagging `random.random() < x` wherever it appears, although most such expressions are benign sampling, jitter, or feature flags. The discriminating question is whether the guarded branch performs a dangerous action, not whether randomness is present. Raising confidence only when a reachable execution or exfiltration sink lies inside the guard, and suppressing it to Low otherwise, keeps genuine bombs at High while dropping benign sampling to a non-gating note.

Worked examples. Three inputs make the behaviour concrete. For `if random.random()<0.14: os.system('rm -rf /')`, the sink scan records `os.system` inside a gated branch,

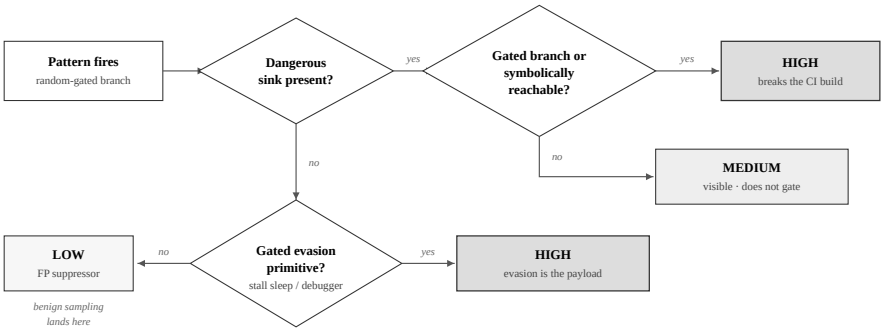

Fig. 2. Confidence assignment. A fired pattern reaches High confidence only when a dangerous sink is reachable inside the gated branch, or is proven reachable symbolically. A sink elsewhere in the file yields Medium. A pattern with no reachable payload yields Low, the level into which benign sampling and feature-flag code falls. A gated anti-analysis primitive is treated as the branch payload and routed to High.

so g_D holds and $c = \text{High}$; by (2) the finding gates. For `if random.random()<0.1: log_metric('b')`, the same pattern fires but no dangerous sink lies in the branch, so $c = \text{Low}$ and the build stays green. For `if random.random()<0.09: time.sleep(99999)`, there is no classic sink, yet the gated stalling sleep is recorded as an evasion payload; g_E holds and the High-severity bomb view gates, the behaviour added in Section 6.

Context-aware down-grading. A vulnerability in a test fixture, example, or documentation file is usually intentional. Q-Trace keeps such findings visible but forces them to Low confidence when the file path matches a test or example context, which prevents alerts on the test trees that most repositories ship.

6 Evidence-Based Anti-Analysis Detection

Anti-analysis behaviour, catalogued as MITRE ATT&CK T1497 [3], is a strong indicator of malice, because benign code rarely checks whether it is being observed. The difficulty is detecting it precisely. A rule that flags any call named `sleep` or any identifier containing "debug" is both over- and under-inclusive: it flags ordinary `log.debug()` logging and short retry sleeps, yet misses debugger detection through `sys.gettrace()`. Q-Trace raises an anti-analysis signal on two evidence classes only. The first is debugger or tracer detection, a call to `sys.gettrace` or `sys.settrace`, which is always flagged. The second is a conditionally gated stalling sleep, a `time.sleep(N)` with a large constant ($N \geq 600$ s) inside a conditional branch; an unconditional long sleep in a daemon loop is not evasion, so gating on the branch preserves precision. When a stalling sleep lies inside a random-gated branch, the construct is both a probabilistic logic bomb and an anti-analysis payload. The evasion primitive is treated as the branch

Algorithm 2 Per-instance confidence assignment.

Require: pattern p; sinks S, some flagged *evasion*; symbolic flag u

$D \leftarrow \{x \in S : \neg x.evasion\}$; $E \leftarrow \{x \in S : x.evasion\}$
$g_D \leftarrow \exists x \in D : x.gated$; $g_E \leftarrow \exists x \in E : x.gated$
if $p =$ STEGO **then** $c \leftarrow$ **High if** $D \neq \emptyset$ **else Medium**
else if $p =$ ANTIDEBUG **then** $c \leftarrow$ **High if** g_E **elif** $E \neq \emptyset$ **Medium else Low**
else if g_D **then** $c \leftarrow$ High
else if $D \neq \emptyset$ **then** $c \leftarrow$ Medium
else if g_E **then** $c \leftarrow$ High ▷ gated evasion is the payload
else if $E \neq \emptyset$ **then** $c \leftarrow$ Medium
else $c \leftarrow$ Low ▷ benign sampling lands here
end if
if $u \wedge c =$ Medium **then** $c \leftarrow$ High ▷ reachability bump
end if
return c

payload, so the two-axis model elevates it to High confidence in the same way as a real sink, and the sleep is never reported as a CWE-78 execution sink.

Figure 5(b) quantifies the change against the prior version of the engine on the same corpus. CI-gate recall rises from 93.5% to 96.8% and F_1 from 0.967 to 0.984; two false-positive classes are removed; and debugger-detection coverage changes from absent to present, with the false-positive rate unchanged at zero. The refinement ships with five regression tests.

7 Implementation

Q-Trace is implemented in Python. It has two dependencies for full function, `numpy` for the auxiliary risk channel and `z3-solver` for symbolic reachability, both treated as optional by the self-healing layer. Parsing uses the standard-library `ast` module, with a token-based fallback for source that does not parse, so the scanner still returns line-anchored findings on inputs that defeat a strict parser. The CI gate of (2) is a short predicate:

```
def is_ci_alert(findings):
    return any(f.severity in ("Critical", "High")
               and f.confidence != "Low" for f in findings)
```

8 Evaluation

8.1 Corpus and methodology

The corpus contains 65 samples, 31 malicious and 34 benign. The malicious samples are reconstructions of techniques documented in public incident write-ups (W4SP, the Hades install hook, a WAV-XOR loader, aiocpa, slopsquatting,

Table 2. Same-corpus comparison at each tool's recommended CI gate. External tools are scored only on inputs they can process; F_1 is on the shared code corpus.

Tool	Malware recall	False-positive rate	F_1
Q-Trace	$30/31 = 96.8\%$	$0/34 = 0.0\%$	**0.984**
Semgrep	$19/29 = 65.5\%$	$1/32 = 3.1\%$	0.776
Bandit	$15/29 = 51.7\%$	$2/32 = 6.2\%$	0.652

environment keying, and an AI-scanner-evasion campaign). We use reconstructions for two reasons of research integrity: the original binaries are neither redistributable nor fetchable offline, and a reconstruction lets a reader inspect exactly what is detected. The benign samples are hard negatives, that is, everyday code on which SAST tools frequently raise false alarms: safe `subprocess` with a list argument, `md5` marked `usedforsecurity=False`, `random` for sampling, parameterised SQL, `verify=True`, character formatting, CI environment checks, and legitimate dependencies.

Each tool runs at its recommended gate: Bandit at MEDIUM or higher severity, Semgrep at WARNING or higher, and Q-Trace at High or higher severity with confidence above Low. Because Bandit and Semgrep cannot process dependency manifests, those corpus rows are excluded from their denominators; scoring a tool on inputs it cannot read would be misleading. Semgrep runs against an offline reimplementation of the standard public Python-security rules, because its registry is unreachable in an air-gapped setting. On the classic classes the tools tie by construction, which is the intended baseline; separation appears only on the classes that need cross-file taint, dependency intelligence, or secret-to-sink correlation.

8.2 Results

Table 2 and Fig. 3 summarise the comparison. Q-Trace detects the correct threat category for all 31 malicious samples and issues a build-breaking alert for 30 of them at a 0% false-positive rate (precision 100%, $F_1 = 0.984$). Semgrep reaches 66% recall at 3.1% false positives, and Bandit 52% at 6.2%. Figure 4 (left) gives the Q-Trace confusion matrix, and (right) shows the separation the confidence axis produces: malicious samples concentrate at High confidence and benign samples at Low or at no finding, with no benign sample reaching the gate.

8.3 Ablation

To isolate the confidence axis, we compare two gates on the same corpus: a severity-only gate that fires on any High or Critical finding, and the two-axis gate of (2). Figure 5(a) reports the outcome. The severity-only gate reaches 100% recall but a 5.9% false-positive rate, because two benign hard negatives break the build. The two-axis gate reduces the false-positive rate to zero at the cost of one borderline detection (recall 96.8%). For a CI gate this trade is favourable,

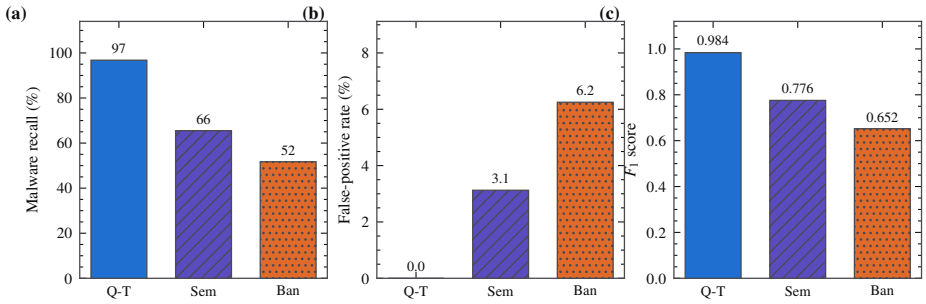

Fig. 3. Same-corpus comparison at each tool's recommended gate: (a) malware recall, (b) false-positive rate on benign hard negatives, and (c) F_1. Bars are hatched for grayscale legibility.

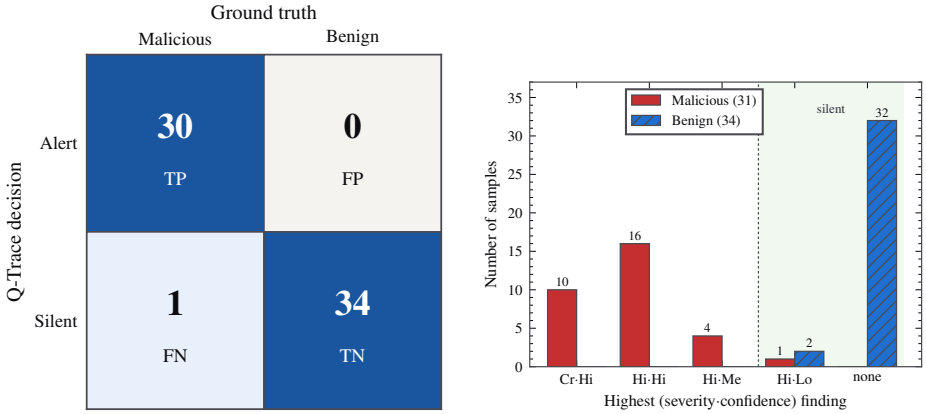

Fig. 4. Left: Q-Trace confusion matrix over all 65 samples (TP=30, FN=1, FP=0, TN=34). Right: distribution of each sample's highest (severity, confidence) finding; no benign sample reaches the gate.

because the eliminated false positives are the kind that erode trust, whereas the suppressed detection remains visible as a Medium or Low note.

8.4 Per-category coverage and reported misses

Figure 6 gives the per-category coverage. Q-Trace detects every category except one. Bandit and Semgrep miss the classes that require cross-file taint, dependency intelligence, or secret-to-sink correlation.

We report the single miss. The SSRF sample `def fetch(u): return requests.get(u)` is held at Low confidence and does not gate. This is deliberate: it is structurally indistinguishable from the benign hard negative `requests.get(u, verify=True)`, since both fetch a function-parameter URL without host validation. Elevating the SSRF finding would also flag the benign case, converting a false negative

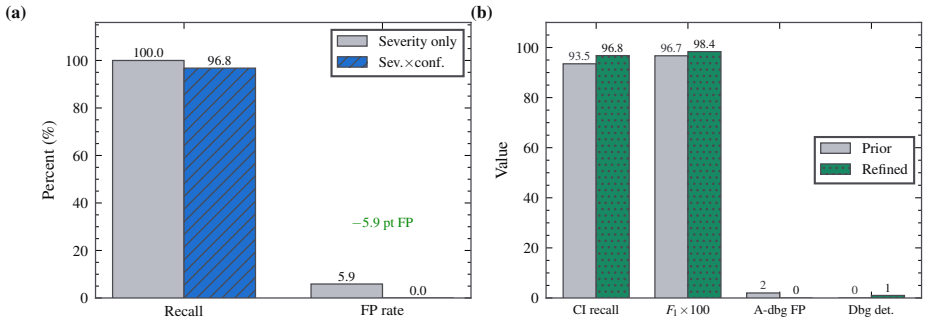

Fig. 5. (a) Confidence-axis ablation: removing the axis recovers one detection but introduces two false positives; the two-axis gate trades 3.2 points of recall for the elimination of all false positives. (b) Effect of the evidence-based anti-analysis refinement on the same corpus.

into a false positive and breaking the zero-false-positive property. Real SSRF is escalated only when taint confirms that the URL is attacker-controlled.

Semantic precision. Coverage counts can favour a tool that fires for an incidental reason. On the credential-exfiltration sample `requests.post(url, data=os.environ)`, Bandit emits a finding, but it is *B113: requests call without a timeout*, not credential exfiltration. An alert that does not name the actual risk is triaged away. Q-Trace instead reports the data flow, `os.environ` to `requests.post`. We therefore separate category recall from gate recall throughout.

8.5 Case studies

Five reconstructed campaigns illustrate that detection follows the structure of the attack, not a signature of a sample. In a W4SP-style stealer split across two files, the interprocedural taint pass links an `os.environ` source in one module to a `requests.post` sink in another and raises credential exfiltration; neither file-by-file tool reconstructs the flow. In a WAV-XOR loader, `exec(bytes([b^42 for b in data]).decode())`, the obfuscation engine measures high entropy and the decode-to-`exec` data path and raises an obfuscated-payload finding. In a Hades-style install hook, top-level `os.system` inside a `setuptools` file raises install-time execution. In an AI-scanner-evasion sample, the prompt-injection scan matches an embedded instruction together with an `os.system` sink. In the stall bomb of Section 6, the gated multi-hour sleep gates the build, whereas the benign daemon `while True: time.sleep(3600)` takes the un-gated path and stays silent. Table 3 generalises the benign side by naming, for each hard-negative class, the discriminator Q-Trace uses to stay silent.

Fig. 6. Per-category coverage. A check denotes a build-breaking alert at the tool's gate, a cross a miss, and a dash an unsupported input class such as a dependency manifest.

8.6 Performance

Across the 28 malicious code samples, median full-audit latency, including taint, classic rules, obfuscation, symbolic reachability, and the auxiliary channel, is 1.8 ms per sample (mean 3.5 ms). The 42 ms maximum is the first sample and reflects one-time solver and library warm-up. Figure 7 shows the distribution. Content-hash caching makes re-analysis of unchanged files free.

9 Discussion

Asymmetry of errors. A CI gate is not symmetric in its error costs. A false negative is a missed detection that other controls, such as code review or dependency scanning, may still catch, and it leaves a visible note for a reviewer. A false positive breaks the build on correct code, and after a few occurrences it reduces trust in the gate, after which developers ignore its output [14,15]. The cost of a false positive is therefore systemic. This asymmetry justifies the design

Table 3. Discriminators that keep Q-Trace silent on benign hard negatives that trip token-level rules.

Benign pattern	Token-level trigger	Discriminator
`subprocess.run(['make'])`	any subprocess call	list arg, no `shell=True`
`md5(b,usedforsecurity=False)`	md5 used	explicit non-security flag
`random.random()<0.1: log()`	randomness present	no reachable sink in branch
`requests.get(u,verify=True)`	dynamic URL	verification not disabled
`log.debug('x')`	"debug" substring	not a tracer or stall primitive
`while True: sleep(3600)`	long sleep	unconditional, not gated
`yaml.safe_load(s)`	yaml load	safe loader in use
test/example fixtures	vulnerability present	path-context down-grade

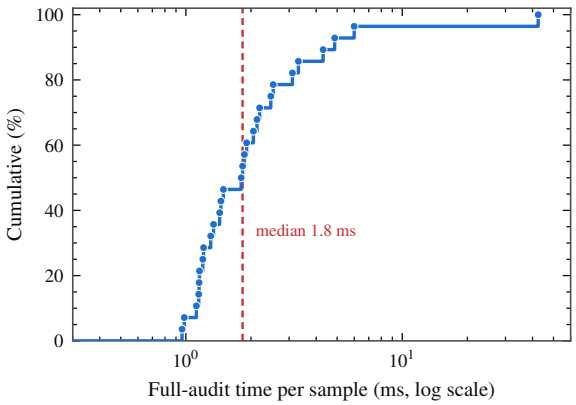

Fig. 7. Empirical cumulative distribution of per-sample full-audit latency (28 malicious code samples, log scale). The dashed line marks the median. The single tall step is cold-start warm-up; steady-state analysis is below 2 ms.

choice to keep an uncertain detection visible but non-gating, as in the SSRF case of Section 8.4.

Determinism. A recent alternative hands source to a language model and asks whether it is malicious. The AI-scanner-evasion technique in our corpus shows the failure mode: an instruction embedded in a comment can steer a model into approving the code. A deterministic analyser has no instruction to override, and it treats the presence of such text as an indicator of malice. Determinism also gives reproducibility and air-gap operation, which a model-based pipeline cannot match. We see language models as suited to explanation rather than adjudication.

Generality. Although demonstrated on Python, the two-axis principle is not language-specific. Wherever a detector faces a trade between coverage and trust, separating impact from reachability-driven evidence and gating on their conjunc-

tion provides a way to raise confidence selectively. The discriminating question in each setting is whether a dangerous effect is reachable, not whether a pattern appears.

10 Limitations and Threats to Validity

Construct validity. The malicious samples are reconstructions, not captured live malware, which improves inspectability but means that recall is measured against our understanding of each technique rather than against an adaptive adversary. A determined adversary can move a dangerous action out of a statically visible gated branch or into a run-time-only path. This limitation is a consequence of undecidability: no static analyser can decide arbitrary run-time behaviour [47], so every such tool trades completeness for precision at some boundary.

Internal and external validity. The benign corpus, although drawn from patterns that empirically cause false positives, is finite (34 samples) and curated by the authors; a different benign distribution could surface false positives we do not observe. The corpus of 65 samples demonstrates the mechanism and the trade decisively for these inputs, but it is not a population-scale study. The 600-second stalling-sleep threshold is an engineering choice, reported so that it can be scrutinised.

11 Ethical Considerations

This work is defensive. Q-Trace detects malicious and vulnerable code and does not generate or improve malware. The malicious corpus consists of minimal reconstructions of already-public techniques, sufficient to exercise a detector but not to serve as a deployable payload, and no live malware is redistributed. The tool runs locally and air-gapped, so scanning proprietary source discloses nothing to third parties. The techniques it detects are documented in public incident reports and in MITRE ATT&CK.

12 Conclusion

We argued that the governing constraint for a CI-deployed security scanner is actionability rather than coverage, because a false positive can disable the tool. The proposed answer is a two-axis, sink-aware confidence model whose CI gate is the conjunction of impact and reachability-driven evidence. On a transparent 65-sample corpus the model reaches 96.8% gate recall at a 0% false-positive rate ($F_1 = 0.984$), and an ablation attributes the elimination of a 5.9% false-positive rate to the confidence axis. An evidence-based anti-analysis detector improves precision and coverage together. Future work includes escalating ambiguous classes such as SSRF automatically when interprocedural taint confirms attacker-controlled input, extending the cross-file taint model to deeper call graphs, and growing the corpus toward a continuously-updated benchmark.

Table 4. Full Q-Trace threat catalogue (28 rules).

Title	CWE	Severity	Base conf.
Credential / Data Exfiltration	CWE-200	Critical	Medium
Cross-Function Embedded Malicious Code	CWE-506	Critical	Medium
Install-Time / Import-Time Code Execution	CWE-506	Critical	Medium
Encoded / Obfuscated Payload	CWE-506	Critical	Medium
Steganographic / Covert Data Channel	CWE-515	Critical	Medium
AI-Scanner Evasion (Prompt Injection in Code)	CWE-506	High	Medium
Chained / Stateful Logic Bomb	CWE-511	High	Medium
OS Command Injection	CWE-78	High	Medium
Dangerous Execution Sink	CWE-78	High	High
Disabled TLS Validation	CWE-295	High	High
Entangled (Multi-Condition) Logic Bomb	CWE-511	High	Medium
Environment-Keyed Trigger	CWE-506	High	Medium
Exposed Secret / Hard-coded Credential	CWE-798	High	High
Hard-coded Credentials	CWE-798	High	Medium
Insecure Deserialization	CWE-502	High	High
Path Traversal	CWE-22	High	Low
Probabilistic Logic Bomb	CWE-511	High	Medium
SQL Injection	CWE-89	High	Medium
Server-Side Request Forgery	CWE-918	High	Low
Typosquat / Slopsquat Dependency	CWE-829	High	Medium
Cleartext Transmission	CWE-319	Medium	Low
Debug Mode Enabled	CWE-489	Medium	Medium
Insufficiently Random Values	CWE-330	Medium	Medium
Insecure Temporary File	CWE-377	Medium	Medium
Anti-Analysis / Anti-Debug Behaviour	CWE-489	Medium	Medium
Weak Cipher	CWE-327	Medium	Medium
Weak Hash Algorithm	CWE-327	Medium	Medium
XML External Entity (XXE)	CWE-611	Medium	Low

Reproducibility. Every quantitative claim is produced by the released harness. The benchmark, including the comparison against Bandit and Semgrep, is reproduced by `python benchmark.py` and `python benchmark.py --compare`; the test suite is `python test_qtrace.py` (94 tests). The corpus is embedded in the harness so that a reader can inspect what is detected, and the per-sample results are written to a report on every run.

Declarations. *Authorship (CRediT):* Dinesh K, conceptualisation, methodology, software, formal analysis, writing (original draft); H Swetha, validation, data curation, visualisation, writing (review and editing). *Funding:* none. *Conflict of interest:* the authors are affiliated with Voxelta Private Limited, which develops Q-Trace; the harness and corpus are released so that all numbers can be reproduced independently. *AI assistance:* automated coding assistance was used for implementation and figure generation under author supervision, and all claims were verified against the released harness.

A Threat Catalogue

Table 4 lists the 28 rules with fixed severity and base confidence; per-instance confidence is computed by Algorithm 2.

Table 5. Malicious corpus (31 samples): per-tool CI-gate outcome.

Sample	Expected class	Q-T	Sem	Ban
probabilistic_bomb	PROBABILISTIC_BOMB	✓	✓	×
chained_bomb	CHAINED_QUANTUM_BOMB	✓	✓	×
cross_func_bomb	CROSS_FUNCTION_QUANTUM_BOMB	✓	✓	×
stego_chr_xor	QUANTUM_STEGANOGRAPHY	✓	×	×
exec_base64	OBFUSCATED_PAYLOAD	✓	✓	✓
xor_blob_exec	OBFUSCATED_PAYLOAD	✓	✓	✓
cred_exfil_direct	CREDENTIAL_EXFILTRATION	✓	×	✓
ssh_key_exfil	CREDENTIAL_EXFILTRATION	✓	×	✓
install_hook	INSTALL_HOOK	✓	✓	×
env_keying	ENVIRONMENT_KEYING	✓	✓	×
ai_evasion	AI_SCANNER_EVASION	✓	✓	×
sql_injection	SQL_INJECTION	✓	✓	✓
cmd_injection	COMMAND_INJECTION	✓	✓	✓
os_system_fmt	COMMAND_INJECTION	✓	✓	✓
pickle_loads	INSECURE_DESERIALIZATION	✓	✓	✓
yaml_load	INSECURE_DESERIALIZATION	✓	✓	✓
eval_input	DANGEROUS_SINK	✓	✓	✓
antidebug_sleep	QUANTUM_ANTIDEBUG	✓	×	×
hardcoded_secret	HARDCODED_SECRET	✓	✓	×
disabled_tls	DISABLED_CERT_VALIDATION	✓	✓	✓
weak_hash_pw	WEAK_HASH	✓	✓	✓
ssrf	SSRF	×	×	✓
path_traversal	PATH_TRAVERSAL	✓	×	×
xxe	XXE	✓	✓	✓
insecure_random_token	INSECURE_RANDOM	✓	×	×
aws_secret_leak	EXPOSED_SECRET	✓	×	×
private_key_leak	EXPOSED_SECRET	✓	×	×
github_token_leak	EXPOSED_SECRET	✓	✓	×
typosquat_req	TYPOSQUAT_DEPENDENCY	✓	–	–
slopsquat_req	TYPOSQUAT_DEPENDENCY	✓	–	–
cross_file_exfil	CREDENTIAL_EXFILTRATION	✓	×	✓

B Full Per-Sample Results

Tables 5 and 6 give the complete per-sample outcome, reproduced from the harness.

References

1. MITRE Corporation: Common Weakness Enumeration (CWE). cwe.mitre.org
2. OWASP Foundation: OWASP Top 10 and Risk Rating Methodology. owasp.org
3. MITRE Corporation: ATT&CK T1480.001 (Environmental Keying) and T1497 (Sandbox Evasion). attack.mitre.org
4. PyCQA: Bandit, a security linter for Python. github.com/PyCQA/bandit
5. Semgrep Inc.: Semgrep, lightweight static analysis. semgrep.dev
6. GitHub Inc.: CodeQL. codeql.github.com
7. OASIS: Static Analysis Results Interchange Format (SARIF) v2.1.0. OASIS Standard (2020)
8. de Moura, L., Bjørner, N.: Z3, an efficient SMT solver. In: Proc. TACAS (2008)
9. Higuchi, T.: Approach to an irregular time series on the basis of the fractal theory. Physica D 31(2), 277–283 (1988)
10. Shannon, C.E.: A mathematical theory of communication. Bell Syst. Tech. J. 27, 379–423 (1948)

Table 6. Benign hard negatives (34 samples): Q-Trace decision and highest finding.

Sample	CI decision	Top finding
flask_run	✓ clean	–
subprocess_list	✓ clean	–
md5_nonsecurity	✓ clean	–
sha256_ok	✓ clean	–
random_sampling	✓ clean	–
random_ab_test	✓ clean	High/Low
yaml_safe	✓ clean	–
sql_parameterized	✓ clean	–
verify_true	✓ clean	High/Low
requests_const_url	✓ clean	–
chr_formatting	✓ clean	–
encode_decode	✓ clean	–
env_debug_print	✓ clean	–
env_ci_print	✓ clean	–
pickle_dump_only	✓ clean	–
open_config	✓ clean	–
argparse_cli	✓ clean	–
dataclass_code	✓ clean	–
pandas_groupby	✓ clean	–
plain_function	✓ clean	–
legit_requirements	✓ clean	–
legit_pyproject	✓ clean	–
base64_encode_cfg	✓ clean	–
comment_security_word	✓ clean	–
logging_debug	✓ clean	–
tempfile_mkstemp	✓ clean	–
secrets_token	✓ clean	–
ssl_default_ctx	✓ clean	–
class_methods	✓ clean	–
env_to_config	✓ clean	–
random_shuffle	✓ clean	–
ignore_word_comment	✓ clean	–
secret_placeholder	✓ clean	–
secret_from_env	✓ clean	–

11. Ohm, M., Plate, H., Sykosch, A., Meier, M.: Backstabber's knife collection: a review of open source software supply chain attacks. In: Proc. DIMVA (2020)
12. Zimmermann, M., Staicu, C.-A., Tenny, C., Pradel, M.: Small world with high risks: a study of security threats in the npm ecosystem. In: Proc. USENIX Security (2019)
13. Ladisa, P., Plate, H., Martinez, M., Barais, O.: SoK: taxonomy of attacks on open-source software supply chains. In: Proc. IEEE S&P (2023)
14. Johnson, B., Song, Y., Murphy-Hill, E., Bowdidge, R.: Why don't software developers use static analysis tools to find bugs? In: Proc. ICSE (2013)
15. Bessey, A., et al.: A few billion lines of code later: using static analysis to find bugs in the real world. Commun. ACM 53(2), 66–75 (2010)
16. Christakis, M., Bird, C.: What developers want and need from program analysis: an empirical study. In: Proc. ASE (2016)
17. Sadowski, C., Aftandilian, E., Eagle, A., Miller-Cushon, L., Jaspan, C.: Lessons from building static analysis tools at Google. Commun. ACM 61(4), 58–66 (2018)
18. Arzt, S., et al.: FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In: Proc. PLDI (2014)
19. Jovanovic, N., Kruegel, C., Kirda, E.: Pixy: a static analysis tool for detecting web application vulnerabilities. In: Proc. IEEE S&P (2006)

20. Spracklen, J., et al.: We have a package for you! A comprehensive analysis of package hallucinations by code-generating LLMs. arXiv preprint (2024)
21. Brumley, D., Hartwig, C., Liang, Z., Newsome, J., Song, D., Yin, H.: Automatically identifying trigger-based behavior in malware. In: Botnet Detection. Springer (2008)
22. Newsome, J., Song, D.: Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In: Proc. NDSS (2005)
23. Schwartz, E.J., Avgerinos, T., Brumley, D.: All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask). In: Proc. IEEE S&P (2010)
24. Nielson, F., Nielson, H.R., Hankin, C.: Principles of Program Analysis. Springer (1999)
25. von Neumann, J.: Mathematical Foundations of Quantum Mechanics. Princeton Univ. Press (1955)
26. FIRST.org: Common Vulnerability Scoring System (CVSS) v3.1 Specification. first.org/cvss
27. Checkmarx, Phylum, Sonatype: Public threat reports on the W4SP stealer, aiocpa, and PyPI install-time payloads (2023–2025)
28. GitGuardian: The State of Secrets Sprawl. Annual report (2025)
29. Python Packaging Authority: pip-audit. github.com/pypa/pip-audit
30. Google: OSV-Scanner and the OSV vulnerability database. osv.dev
31. King, J.C.: Symbolic execution and program testing. Commun. ACM 19(7), 385–394 (1976)
32. Cadar, C., Dunbar, D., Engler, D.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: Proc. OSDI (2008)
33. Godefroid, P., Klarlund, N., Sen, K.: DART: directed automated random testing. In: Proc. PLDI (2005)
34. Godefroid, P., Levin, M.Y., Molnar, D.: Automated whitebox fuzz testing. In: Proc. NDSS (2008)
35. Fratantonio, Y., Bianchi, A., Robertson, W., Kirda, E., Kruegel, C., Vigna, G.: TriggerScope: towards detecting logic bombs in Android applications. In: Proc. IEEE S&P (2016)
36. Yamaguchi, F., Golde, N., Arp, D., Rieck, K.: Modeling and discovering vulnerabilities with code property graphs. In: Proc. IEEE S&P (2014)
37. Livshits, V.B., Lam, M.S.: Finding security vulnerabilities in Java applications with static analysis. In: Proc. USENIX Security (2005)
38. Lyda, R., Hamrock, J.: Using entropy analysis to find encrypted and packed malware. IEEE Secur. Priv. 5(2), 40–45 (2007)
39. Chess, B., McGraw, G.: Static analysis for security. IEEE Secur. Priv. 2(6), 76–79 (2004)
40. Ayewah, N., Pugh, W., Hovemeyer, D., Morgenthaler, J.D., Penix, J.: Using static analysis to find bugs. IEEE Softw. 25(5), 22–29 (2008)
41. Distefano, D., Fähndrich, M., Logozzo, F., O'Hearn, P.W.: Scaling static analyses at Facebook. Commun. ACM 62(8), 62–70 (2019)
42. Duan, R., Alrawi, O., Kasturi, R.P., Elder, R., Saltaformaggio, B., Lee, W.: Towards measuring supply chain attacks on package managers for interpreted languages. In: Proc. NDSS (2021)
43. Vu, D.-L., Massacci, F., Pashchenko, I., Plate, H., Sabetta, A.: LastPyMile: identifying the discrepancy between sources and packages. In: Proc. ESEC/FSE (2021)
44. Sejfia, A., Schäfer, M.: Practical automated detection of malicious npm packages. In: Proc. ICSE (2022)

45. Torres-Arias, S., Afzali, H., Kuppusamy, T.K., Curtmola, R., Cappos, J.: in-toto: providing farm-to-table guarantees for bits and bytes. In: Proc. USENIX Security (2019)
46. Enck, W., et al.: TaintDroid: an information-flow tracking system for realtime privacy monitoring on smartphones. In: Proc. OSDI (2010)
47. Rice, H.G.: Classes of recursively enumerable sets and their decision problems. Trans. Amer. Math. Soc. 74, 358–366 (1953)
48. Open Source Security Foundation: SLSA, Supply-chain Levels for Software Artifacts. slsa.dev
49. Sonatype: State of the Software Supply Chain. Annual report (2024)
50. Guo, W., et al.: An empirical study of malicious code in PyPI ecosystem. In: Proc. IEEE/ACM ASE (2023)
51. Scholte, T., Balzarotti, D., Kirda, E.: Have things changed now? An empirical study on input validation vulnerabilities in web applications. Comput. Secur. 31(3), 344–356 (2012)
52. Ohm, M., Kempf, L., Boes, F., Meier, M.: Supporting the detection of software supply chain attacks through unsupervised signature generation. arXiv preprint (2021)